HOMEWORK 5

Context Engineering, Prompt Engineering & Multi-Agent "Vibe Coding"

Guided project: build a *Research Paper Library & Knowledge Graph* with a team of AI models

Course: [CS CRP] **Due:** [23rd October, 2026]

Type: Individual, guided project **Submit:** 4 files (see Section 7)

Assignment at a glance

- **Goal.** Learn how to get excellent software out of AI tools—even a *weak* one—by giving it excellent *context* and excellent *prompts*.
- **What you build.** A web app where you upload research-paper PDFs; it parses them, tags them, stores everything in SQLite, remembers who did what, and draws a knowledge graph connecting papers, authors and topics.
- **How.** A fixed pipeline of free-tier AI tools: **Gemini Deep Research / DeepSeek** (research) → **Perplexity** (tech-stack lookup) → **Claude** (planning) → **Qwen 2.5 Coder 7B** (writes the whole app).
- **Hard constraints.** Python + SQLite backend, vanilla HTML/CSS/JS frontend, *no build step*, *no Node.js/Bun/Deno*, *no npm*, *no JS frameworks*.
- **You submit.** `Deliverable_1_CONTEXT_[HeUserID].md`, `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`, `Deliverable_3_USER_PROMPTS_[HeUserID].md` and `Deliverable_4_APP_[HeUserID].zip`.
- **Expected effort.** Roughly 10 hours spread over one weeks. Start early: free tiers have rate limits.

Contents

1 Why This Assignment Exists **4**

1.1 Learning Objectives 4

2 Background: The Concepts You Must Understand **4**

2.1 Vibe Coding 4

2.2 Chatbot vs. AI Agent 5

2.3 Tokens and the Context Window 5

2.4 Context Engineering 6

2.5 Prompt Engineering 6

2.6 Context Engineering vs. Prompt Engineering 7

2.7 Why We Force a Weak Model at the End 7

3 The Tool Landscape and Your Assigned Roles **8**

4 The Project: Research Paper Library & Knowledge Graph **9**

4.1 The Idea . . . 9

4.2 Why This Is a Perfect Context-Engineering Exercise . . . 9

4.3 Functional Requirements . . . 10

4.4 The Graph Model . . . 10

5 Hard Constraints (Non-Negotiable) **11**

5.1 Overall . . . 11

5.2 Strict Frontend Constraints (No-Build Architecture) . . . 11

5.3 Quick Self-Check Before Submitting . . . 12

6 The Workflow: Four Phases, Four Tools **12**

6.1 Phase 1 — Domain Research and Context Engineering . . . 12

6.1.1 Step 1: Ask the right questions . . . 12

6.1.2 Step 2: Choose and use the right research tool . . . 13

6.1.3 Step 3: Sift (the heart of context engineering) . . . 13

6.1.4 Step 4 (bonus): Ground your research in real samples . . . 14

6.1.5 Required content of `Deliverable_1_CONTEXT_[HeUserID].md` . . . 14

6.2 Phase 2 — Technology-Stack Lookup . . . 15

6.3 Phase 3 — Planning and Prompt Writing with Claude . . . 15

6.3.1 What you give Claude . . . 16

6.3.2 What the plan must contain . . . 16

6.3.3 Checklist for each Qwen prompt . . . 16

6.3.4 Required content of `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md` . . . 17

6.4 Phase 4 — Build the App with Qwen 2.5 Coder 7B . . . 18

6.4.1 The build loop . . . 18

6.4.2 Debugging protocol for a weak model . . . 18

6.4.3 Rules on manual edits . . . 19

6.4.4 Recommended module order . . . 19

6.5 Packaging the App . . . 19

7 Deliverables and Submission **20**

7.1 Required Format of `Deliverable_3_USER_PROMPTS_[HeUserID].md` . . . 20

7.2 Pre-Submission Checklist . . . 21

7.3 Submission Form 21

8 Grading **22**

9 AI-Use and Academic-Integrity Policy **23**

10 Suggested Timeline **23**

11 Frequently Asked Questions and Common Pitfalls **24**

12 Glossary **25**

1 Why This Assignment Exists

Many people now build software by describing what they want to an AI and pasting the result back and forth—often called **vibe coding**. It feels magical for a 100-line game. It falls apart on anything with several files, a database and real-world messy input, because the AI is guessing about things you never told it.

The fix is not "a smarter AI". The fix is **you**, acting like a good engineering lead:

1. You *investigate the problem domain* so you know what the input actually looks like. (**Context Engineering**)
2. You *write precise, structured instructions* so nothing is left to guess. (**Prompt Engineering**)
3. You *choose the right tool for each stage* instead of using one chatbot for everything. (**Multi-agent workflow**)

To prove the point, the last stage of this homework deliberately forces you to build the final app with a **small, weak model** (Qwen 2.5 Coder 7B). If your context and prompts are excellent, even this small model will produce a working app. If they are vague, it will visibly fail—and you will learn exactly why.

1.1 Learning Objectives

After this homework you should be able to:

LO1. Explain the difference between *context engineering* and *prompt engineering*, and when each applies.

LO2. Perform structured domain research and *sift* it into a compact, machine-readable context pack.

LO3. Choose appropriate AI tools (research agents, search LLMs, reasoning models, coding models) for different project phases and justify the choice.

LO4. Select and verify a technology stack using up-to-date, citation-backed sources, avoiding deprecated libraries.

LO5. Write a complete architecture blueprint: file tree, database schema, API contract, build order.

LO6. Write verbose, deterministic prompts that a weak model can execute reliably, and debug failures systematically.

LO7. Respect real engineering constraints (no build step, fixed language/DB) and deliver a runnable, documented project.

2 Background: The Concepts You Must Understand

Read this whole section before touching any AI tool. Your grade depends on applying these ideas, not just on the final app.

2.1 Vibe Coding

Vibe coding is a term popularized in early 2025 by Andrej Karpathy (former AI director at Tesla and OpenAI co-founder). The human stops writing code line by line and acts more like a product manager or director: describe the intent ("the vibe"), run the result, paste errors back, and let the AI write the code.

The trap: pure vibe coding collapses on multi-file projects into spaghetti code and endless bug loops, because the AI keeps forgetting or inventing details. The cure is the two disciplines below.

2.2 Chatbot vs. AI Agent

	Chatbot (conversational LLM)	**AI agent (agentic coding system)**
What it is	A passive, turn-based interface: you send text, it returns text. Examples: web chat UIs of Claude, Gemini, DeepSeek, Perplexity, Qwen.	An LLM connected to *tools, memory and an execution environment*. Examples: Cursor/Windsurf (agent mode), Cline, Claude Code, GitHub Copilot agent features.
Can it touch your files?	No. You copy and paste.	Yes: reads your folder, edits many files, runs terminal commands.
Can it check its own work?	No. It cannot run its code.	Yes: runs the code, reads the error, fixes it in a loop.
In this homework	Most of your tools are chatbots (or search/research agents). *You* are the execution environment: you run the code and report errors.	Research tools such as Deep Research behave agent-like (they browse autonomously), but coding is done through chat.

Terminology note

"GPT" (Generative Pre-trained Transformer) is OpenAI's product branding. The correct umbrella term for DeepSeek, Gemini, Claude, Qwen and similar systems is **LLM (Large Language Model)**. Models that deliberately "think" step by step before answering are often called **reasoning models**.

2.3 Tokens and the Context Window

- **Token:** the unit of text an LLM reads and writes. Roughly 1 token $\approx$ 4 characters $\approx$ 0.75 English words.
- **Context window:** the model's working memory for *one session*: your prompt, attached files, the whole chat history *and* the model's own replies must all fit inside it.
- **Different models, very different windows.** Gemini's long-context models are at the top of the public range (hundreds of thousands up to around a million tokens or more, depending on the version and where you use it, e.g. Google AI Studio). DeepSeek's chat models are in the tens-to-low-hundreds of thousands. Free web tiers often cap you well below a model's advertised maximum. Small models such as Qwen 2.5 Coder 7B may have large advertised limits, but in practice (especially when run locally or via free hosts) you should treat them as having a *small effective window* and keep every prompt short and focused.
- **Context rot / "lost in the middle":** the more irrelevant material you pour in, the worse the model attends to your actual instructions. Instructions buried in the middle of a huge prompt are the first to be ignored. *More context is not better context.*

2.4 Context Engineering

Definition. Context engineering is the systematic practice of *curating, structuring, compressing and sequencing* the information placed in an LLM's context window—domain rules, documentation, data schemas, examples, file trees, constraints—so that the model can reason accurately instead of guessing.

Analogy. If prompt engineering is *how you phrase the question*, context engineering is *the open-book study guide you hand over before asking it*.

Why you can't skip it. You cannot design an algorithm or a tool without knowing what its *input* looks like. A program that ingests research papers must be designed around how research papers are actually structured, how PDFs actually store text, and how citations are actually written. If you don't know these things, neither does the AI—it will invent plausible nonsense (a *hallucination*).

The four pillars

1. **Discovery & retrieval.** Gather raw domain truth: what is in a research paper? How are PDFs built? Which free APIs return clean metadata? Which Python libraries parse PDFs well? (Tools: Gemini Deep Research, DeepSeek, Perplexity.)
2. **Selection & pruning (sifting).** Throw away fluff, history, marketing, and irrelevant edge cases. Keep only what changes a design or coding decision.
3. **Structuring.** Convert messy notes into clean, machine-readable reference material: Markdown specs, JSON examples, SQL schemas, tables, short code snippets.
4. **State management.** Maintain a small "memory bank" (your `CONTEXT.md`, architecture notes, a running "what is done / what is next" list) that you paste at the start of each new AI session so it instantly knows the project's state.

2.5 Prompt Engineering

Definition. Prompt engineering is the tactical craft of structuring instructions, roles, constraints and output formats so an LLM produces a predictable, high-quality result.

Analogy. Once the AI has the background (context), the prompt is the *work order or ticket*. A great ticket leaves zero room for guessing.

Anatomy of a good engineering prompt

1. **Role & objective** — *"You are a senior Python backend engineer. Write the PDF ingestion module."*
2. **Context reference** — the exact facts needed (pasted in, since most chat tools cannot read your files): schema, API contract, sample input/output.
3. **Tech stack & constraints** — exact language version, libraries, and prohibitions (*"Do NOT use PyPDF2; do NOT use Node or npm"*).
4. **Interface contract** — exact file name, function signatures, input and output types.
5. **Step-by-step plan** — numbered steps the model should follow.
6. **Output format** — *"Return only the complete contents of `parser.py` in one code block. No explanation."*
7. **(Optional) Examples** — one small input and the exact expected output (few-shot).

2.6 Context Engineering vs. Prompt Engineering

Dimension	Context engineering (the environment)	Prompt engineering (the command)
Core question	"What does the AI need to *know* to do this job?"	"What exact *task* do I want it to do right now?"
Project phase	Research, discovery, architecture setup	Active coding, debugging, iteration
Typical artifacts	Domain notes, API docs, schemas, sample files, constraint lists	Step-by-step instructions, contracts, output-format rules, examples
If you skip it	**Hallucination:** fake libraries, wrong assumptions about input, broken integrations	**Ambiguity:** wrong language, sloppy or partial code, vague answers

2.7 Why We Force a Weak Model at the End

A frontier model can often recover from a vague prompt by guessing correctly. A 7B-parameter model cannot. That makes it an excellent *measuring instrument*: if Qwen 2.5 Coder 7B builds your app, it is because your context and prompts were good. Expect to learn these habits the hard way:

- One small task per prompt; one file (or even one function) at a time.
- State the exact interface (names, parameters, return types) so independently generated files fit together.
- Paste only the context *relevant to that task*, not your whole research.
- Re-state the hard constraints in every prompt; small models forget them.
- Start a *fresh chat* when the conversation gets long or the model starts to drift.

3 The Tool Landscape and Your Assigned Roles

Different AI tools are good at different jobs. This homework assigns a tool to each phase. Use the assigned tools; the point is to learn *why* each fits its role.

Tool	Strengths	Weaknesses	Role in this homework
Gemini (Deep Research; long-context models)	Browses the web autonomously and compiles long cited reports. Very large context window: can read many PDFs at once.	Can over-engineer if not constrained. Free-tier Deep Research usage is limited.	**Phase 1:** domain research; digesting sample papers.
DeepSeek (chat, with reasoning mode)	Free; strong step-by-step reasoning for logic, algorithms and architecture.	Slow, because it *thinks* before answering (not because of memory). Smaller window than Gemini. Servers are sometimes busy.	**Phase 1:** alternative or complement to Gemini for research and reasoning.
Perplexity	Fast, citation-backed lookups of current library versions, APIs and documentation.	Small context; poor at writing large multi-file code.	**Phase 2:** choosing libraries and checking they are current and not deprecated.
Claude (free tier)	Excellent at structured, well-organized, detailed writing and planning.	Free tier has a small message/token budget that runs out quickly.	**Phase 3:** turn your context and stack into the full plan and prompts. *Do not waste it on research.*
Qwen 2.5 Coder 7B (Alibaba; free tier or local)	Free, code-specialized, surprisingly capable when given precise instructions.	Small model: forgets constraints, weak at long reasoning, easily confused by vague prompts.	**Phase 4 (mandatory):** writes *all* application code.

Tool usage rules

- **Do not use Claude in Phase 1 or Phase 2.** Its free-tier budget is limited; save it for planning.
- **All application code must come from Qwen 2.5 Coder 7B (Instruct variant).** Do not use Claude, ChatGPT, Gemini, Copilot or any other model to write or fix application code. (Manual edits are allowed in a limited, logged form; see Section 6.4.)
- Free tiers, model names and limits change often. If an assigned tool is unavailable to you, tell the instructor *before* substituting, and record the substitution in `Deliverable_3_USER_PROMPTS_[HeUserID].md`.
- You may run Qwen through a free hosted chat/API provider, or locally (for example with Ollama: `ollama run qwen2.5-coder:7b`, or LM Studio) if your laptop can handle a 7B model. Record which route you used and the exact model name.

4 The Project: Research Paper Library & Knowledge Graph

4.1 The Idea

You are building a personal **digital library for research papers**. A user uploads a paper as a PDF. The app:

1. parses it and extracts the title, authors, abstract, keywords, references and other useful metadata;
2. automatically assigns **tags** and **subject matters** (topics);
3. stores everything in a **SQLite** database, including a record of *who uploaded or changed what, and when*;
4. lets the user browse and search the library;
5. as more papers are added, draws an interactive **knowledge graph** in which papers, authors and topics are connected to each other.

4.2 Why This Is a Perfect Context-Engineering Exercise

If you simply tell an AI "make an app that reads PDFs and graphs them", it will fail. PDFs are not structured documents like HTML or Word; they are drawing instructions that place glyphs at coordinates on a page. Academic papers add more difficulty: two-column layouts, headers and footers, equations, ligatures, hyphenated line breaks, and many different citation styles. Before you can design *any* parser you must understand what a research paper looks like from the inside. That is your Phase 1 research.

4.3 Functional Requirements

ID	Requirement	Priority
F1	Upload a PDF through the browser; the file is stored on the server and a record is created.	Must
F2	Parse the PDF and extract at least: title, author list, abstract. Extract keywords, DOI/arXiv ID, year and venue where available.	Must
F3	Extract the reference list and store each reference as a separate record (raw text at minimum; structured fields if you can).	Must
F4	Automatically assign tags/subjects to each paper (see ideas in Section 6.1). Tags must be stored in the database.	Must
F5	Persist everything in SQLite with sensible relational tables (papers, authors, topics, references, links between them).	Must
F6	Record who did what: every upload, edit, tag change and deletion is logged with a user name, action and timestamp. A simple "enter your name" or minimal registration is sufficient; full password security is not required.	Must
F7	Library view: list papers, search/filter by title, author, topic or tag.	Must
F8	Paper detail view: show the extracted metadata, abstract, tags and references; allow correcting metadata by hand.	Should
F9	Knowledge graph view: interactive graph with at least three node types (paper, author, topic) and meaningful edges. It must update as papers are added.	Must
F10	Activity/history view: show the log from F6.	Should
F11	Delete a paper (and keep the graph and database consistent).	Should
F12	Handle bad input gracefully: non-PDF files, scanned PDFs without text, duplicate uploads. Show clear error messages instead of crashing.	Should

4.4 The Graph Model

Your research and plan must make these concepts precise. A starting point:

- **Nodes:** *Paper*, *Author*, *Topic* (optionally *Venue* or *Keyword*).
- **Edges:** `AUTHORED_BY` (paper–author), `HAS_TOPIC` (paper–topic), `CITES` (paper–paper, only when the cited paper is also in the library), and derived edges such as `CO_AUTHOR` (author–author) and `RELATED_TOPIC` (topic–topic, e.g. topics that co-occur in the same papers).
- **Design question to answer:** Do you store the edges in tables, or compute them from existing tables when the graph is requested? Justify your choice in `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`.

Hint: don't parse everything from the PDF alone

Messy PDF text is a fragile source of truth. A smart engineer extracts a stable identifier (a DOI or arXiv ID, or at worst a clean title) and asks a free scholarly metadata service for *clean JSON* (for example Crossref, Semantic Scholar, OpenAlex or the arXiv API). Investigate these

services in Phase 1: what do they return, what are their rate limits and licensing terms, and what happens when a paper has no DOI? Your app should still work (in a degraded way) when the network or the lookup fails.

5 Hard Constraints (Non-Negotiable)

Violating any of these costs you heavily (see grading)

These apply to the *final app*. Re-state them in every prompt you give to Claude and Qwen.

5.1 Overall

- The project must be **web-based**.
- **Backend language: Python. Database: SQLite.** You may use any number of Python libraries (installed with `pip` and listed in `requirements.txt`).
- **No build process** for the frontend *or* the backend.
- **Strictly forbidden:** Node.js, Bun, Deno, and any JavaScript backend; `npm` (and `yarn`, `pnpm`); any `package.json` or `node_modules` in your project.

5.2 Strict Frontend Constraints (No-Build Architecture)

1. **Environment:** browser-native client-side code only: HTML5, CSS3 and vanilla ES6+ JavaScript.
2. **No Node.js:** do not generate a `package.json`; do not use `npm` or `node_modules`; do not use build tools (no Vite, Webpack, Babel or TypeScript).
3. **No frameworks:** no React, Vue, Angular or JSX. Manipulate the DOM directly with standard Web APIs (`document.querySelector`, `fetch`, etc.).
4. **External libraries via CDN only:** if you need third-party libraries (for example `vis-network` or `cytoscape` for graphs, `pdf.js` for PDFs), load them strictly through `<script src="https://cdn.jsdelivr.net/...">` tags in `index.html`, or through browser-native ES-module URL imports (`import ... from 'https://esm.sh/...'`).
5. **Execution:** the frontend must work immediately when opened through a static file server or directly from `index.html`.

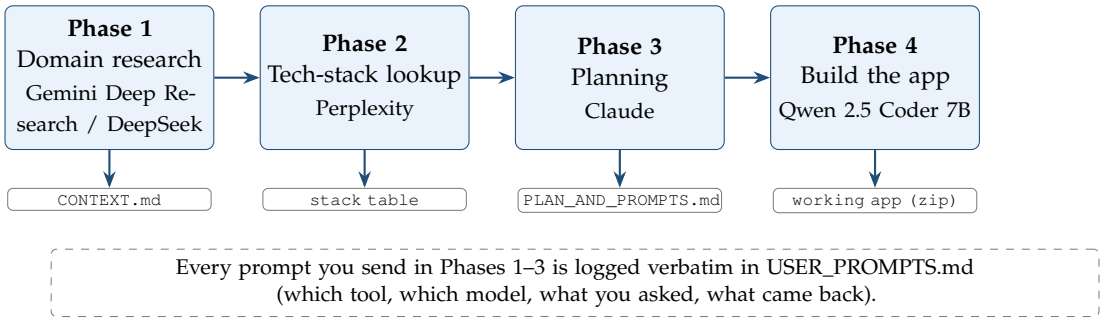
Engineering detail you should research: serving and CORS

If your page is opened straight from disk (`file://`) and calls your Python server with `fetch`, the browser will block the request unless the backend sends CORS headers. If instead your Python server also serves the static frontend files, there is no CORS problem. Decide which approach you use, say so in your plan, and make sure the app works with the exact instructions in your README. Also pin your CDN library versions in the URLs so the app does not break when a library updates.

5.3 Quick Self-Check Before Submitting

```
# From the project root (inside the unzipped folder), these must find nothing:
find . -name package.json -o -name node_modules -o -name "*.jsx" -o -name "*.tsx" -o -
    name "*.ts"
grep -rniE "from 'react'|require\(|webpack|vite|babel" --include=*.js --include=*.
    html .
```

6 The Workflow: Four Phases, Four Tools



The golden rule of this pipeline

The output of each phase is the input of the next. Do not paste raw chat transcripts forward; paste a *cleaned, structured artifact*. Quality compounds: good context → good plan → good prompts → working code.

6.1 Phase 1 — Domain Research and Context Engineering

Goal: understand the *input* (research-paper PDFs) and the *ecosystem* (libraries, APIs, techniques) well enough that you could explain exactly what the program must do. **Tools:** Gemini Deep Research and/or DeepSeek. **Output:** `Deliverable_1_CONTEXT_[HeUserID].md`.

6.1.1 Step 1: Ask the right questions

Do not ask one giant question like "how do I build this app?". Run several focused research sessions. Your research must at least answer the following.

- **Q1. Anatomy of a research paper.** Which sections exist, in what order, and what do they contain (title block, authors and affiliations, abstract, keywords, introduction, methods, results, discussion, conclusion, acknowledgements, references)? How do conference papers, journal papers and arXiv preprints differ?
- **Q2. The PDF problem.** How does a PDF actually store text? Why do two-column layouts, headers/footers, footnotes, equations, ligatures and hyphenation break naive text extraction? What is the difference between a text PDF and a scanned PDF?
- **Q3. Citation and reference styles.** What do IEEE (numbered), APA, MLA, Chicago and BibTeX-style references look like? How are in-text citations different from the bibliography? Give a *real* example of each.
- **Q4. Identifiers and metadata services.** What are a DOI and an arXiv ID, what do they look like (patterns), and where do they appear in a PDF? Which free scholarly APIs can turn them into clean metadata? What fields do they return, and what are the rate limits and terms?

Q5. Extraction techniques. Which Python libraries extract text and layout from PDFs (and what are their trade-offs)? Which approaches find the title, authors, abstract and reference section (heuristics such as font size and position, regular expressions, section-heading detection, metadata lookup, or an LLM)?

Q6. Tagging and topic assignment. Which techniques can assign keywords/topics automatically using only free and local tools (for example keyword extraction such as RAKE/YAKE/TF–IDF, using the paper's own keywords, matching against a small controlled vocabulary, or topic information returned by a scholarly API)? What are the strengths and limits of each?

Q7. Knowledge graphs. What is a graph in the data-structure sense (nodes, edges, directed vs. undirected)? How do you represent one in relational tables? Which browser libraries draw interactive graphs from a CDN, and what JSON format do they expect?

Q8. Edge cases. What goes wrong in real papers: missing abstract, multi-line titles, many authors, non-English text, no DOI, scanned pages, references split across columns, duplicate uploads?

6.1.2 Step 2: Choose and use the right research tool

- **Gemini Deep Research**: give it a clear, scoped brief. It browses many sources and returns a long cited report. Check the citations; do not trust every claim.
- **DeepSeek**: use it to reason through design questions, compare approaches, and cross-check Gemini's report. Expect slower answers.
- **Use both when possible** and compare. Where they disagree, find out which is right. Noting a disagreement you resolved is a mark of strong research.

Starter research prompt (adapt it; do not copy it blindly)

```
I am designing a program that ingests academic research papers as PDF
files and extracts structured metadata (title, authors, abstract,
keywords, references). I have not yet chosen any design. Research and
report on: (1) the typical structure of conference, journal and arXiv
papers; (2) why PDF text extraction is hard for two-column papers and
how Python libraries handle it; (3) the main citation styles with one
real example each; (4) DOI and arXiv ID formats and free metadata APIs
with their response fields and rate limits. Cite every source. Prefer
concrete examples and short sample JSON over general prose.
```

6.1.3 Step 3: Sift (the heart of context engineering)

Raw research reports are long, repetitive and full of noise. Your job is to compress them into what the downstream AI actually needs.

Keep (changes a design or code decision)	Throw away (noise)
Concrete formats, patterns and examples (regexes for DOI, a real IEEE reference line, a real API JSON response, trimmed)	History of the PDF format, publishing-industry politics, marketing language
Library names with a one-line trade-off each	Long tutorials, installation walk-throughs, unrelated libraries
Edge cases that will actually break your parser	Exotic cases outside your scope (e.g. training your own OCR model)
Precise definitions of your graph's nodes and edges	Generic explanations of what a graph is
Sources (URLs) so claims can be re-checked	Unsourced claims you could not verify

Aim for a context pack small enough to paste into a model with a modest window *together with* other instructions: roughly **1 500 to 3 500 words**, dense and well structured. In `Deliverable_1_CONTEXT_[HeUserID].md`, include a short "**What I discarded and why**" section; this shows your sifting judgement.

6.1.4 Step 4 (bonus): Ground your research in real samples

Extra credit. Download **three to five real research papers** (open-access ones: for instance from arXiv; choose a mix, e.g. one two-column conference paper, one single-column paper, one with many authors). Give them to Gemini (it can ingest several PDFs at once) and ask it to extract the title, authors, abstract and references, and to describe the exact patterns it sees (where the DOI appears, how the reference section starts, how references are numbered). Put the *observed* patterns and a few short, real excerpts into your context pack. Real samples beat imagination; this is the single best way to stop the coding model from hallucinating about the input. Evidence of this step (the prompt, the sample list, and the findings) is required to receive the bonus.

Respect copyright

Use papers you may legally download (open access/preprints). In your submission include only short excerpts needed for illustration, not whole papers.

6.1.5 Required content of `Deliverable_1_CONTEXT_[HeUserID].md`

```
# CONTEXT - Research Paper Library & Knowledge Graph
## 1. Project in one paragraph
## 2. Anatomy of a research paper (sections, order, what each contains)
## 3. The PDF problem (how text is stored; two-column, scanned, etc.)
## 4. Citation styles (IEEE / APA / MLA / BibTeX) - one REAL example each
## 5. Identifiers and metadata APIs
   - DOI / arXiv ID patterns and where they appear
   - For each API: endpoint, auth, rate limit, trimmed sample JSON
## 6. Extraction techniques considered -> which one I chose and why
## 7. Tagging / topic assignment techniques -> chosen approach
## 8. Knowledge-graph model (nodes, edges, direction, derived edges)
## 9. Edge cases and failure modes the app must handle
## 10. Sample-paper findings (bonus, if done)
## 11. What I discarded and why (sifting log)
## 12. Sources (URLs)
```

6.2 Phase 2 — Technology-Stack Lookup

Goal: pick specific, *current*, non-deprecated libraries for each need and record exact versions. **Tool:** Perplexity. **Output:** the technology table that goes into `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`.

Why Perplexity? Coding models are trained on data that is months or years old, so they happily use functions that have been renamed, deprecated or removed. Perplexity answers with live web sources and citations, which makes it ideal for "is this library still maintained, and what is its current API?" (It has a small effective context, so use it for short, targeted lookups only—not for generating your project.)

What to look up. For each row, ask a precise question, open the cited source, and confirm it yourself:

Need	Questions to answer
Python web framework	Which lightweight framework (e.g. Flask, FastAPI, or another) fits a small app that also serves static files and accepts file uploads? Current stable version? Any upload or CORS gotchas?
PDF text/layout extraction	Which actively maintained library (e.g. PyMuPDF, pdfplumber, pypdf) best handles two-column papers? Is any option deprecated or unmaintained (e.g. PyPDF2)?
Metadata extraction / APIs	Which HTTP client library? Current endpoint and response format for Crossref / Semantic Scholar / OpenAlex / arXiv? Rate limits?
Tagging / keyword extraction	Which Python library (if any) for local keyword extraction? Is it maintained? Does it need big model downloads?
SQLite access	Use the standard-library `sqlite3` or an ORM? What is the recommended way to avoid SQL injection?
Graph visualization (CDN)	Current stable version of `vis-network`, `cytoscape.js` or similar; the exact jsDelivr URL; the required data format; does it need an extra CSS file?

Record for each choice: library name, **exact version**, purpose, why chosen over alternatives, and the **source URL**. Prefer official documentation or package-index pages over blog posts.

Example lookup prompt

```
What is the current stable version of PyMuPDF, and what is the
recommended way to extract text blocks with positions from a PDF page
in Python? Is PyPDF2 deprecated? Cite the official documentation.
```

6.3 Phase 3 — Planning and Prompt Writing with Claude

Goal: turn your context pack and stack table into a complete blueprint—detailed enough that a 7B model can build the app one small piece at a time. **Tool:** Claude (free tier). **Output:** `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`.

Budget your Claude usage

The free tier allows only a limited amount of text. Prepare *before* you open it: finalize `Deliverable_1_CONTEXT_[HeUserID].md`, your stack table and your constraints list, and write one well-structured planning prompt. Then iterate in a few focused follow-ups. Do not use Claude to explore or research.

6.3.1 What you give Claude

1. Your cleaned context pack (from Phase 1).
2. Your stack table with exact versions (from Phase 2).
3. The functional requirements F1–F12 and the full constraints from Section 5.
4. A description of the executor: "The code will be written by Qwen 2.5 Coder 7B, a small model with a limited effective context. Each prompt must therefore be self-contained, produce *one file or one small group of functions*, and state all interfaces explicitly."

6.3.2 What the plan must contain

1. **Overview & design decisions** (including the graph-storage choice from Section 4.3).
2. **Technology table** (from Phase 2).
3. **Architecture:** a complete **file/folder tree**, the responsibility of each file, and the data flow from upload to graph (a simple diagram or numbered list).
4. **Database schema:** full `CREATE TABLE` statements with primary/foreign keys, uniqueness rules, and indexes; including the activity-log table.
5. **API contract:** every endpoint with method, path, request format, response JSON (with an example) and error responses.
6. **Frontend specification:** pages/views, the DOM structure at a high level, the exact CDN `<script>` tags with pinned versions, and how the graph JSON maps to the visualization library.
7. **Build order:** the sequence of modules, with dependencies (e.g. database before API before parser integration before frontend).
8. **The Qwen prompts:** one numbered prompt per module (Prompt 0, Prompt 1, ...), each independently pasteable. See the checklist below.
9. **Acceptance test for each module:** a concrete command or browser action that proves the module works (e.g. "run `python db.py` and see 5 tables created").

6.3.3 Checklist for each Qwen prompt

Every prompt in your plan must contain:

- ☐ **Role** and one-sentence **objective**.
- ☐ The **hard constraints**, restated (Python, SQLite, no Node/npm/frameworks/build step).
- ☐ The **exact file name and path** to produce.
- ☐ The **interface contract**: function names, parameters, return types, and any shared constants. Files written by separate prompts must fit together.
- ☐ The **minimum necessary context** (e.g. the relevant table definitions, the JSON shape it must return), pasted in.

☐ A short **numbered list of steps**.
☐ The **output format**: "only the complete file in a single code block, no explanation".
☐ An optional tiny **example** input and expected output.

Illustration only: what one good prompt looks like (for a different module than you will write)

```
ROLE: You are a senior Python backend engineer.
OBJECTIVE: Write ONE file, backend/doi_utils.py.

HARD CONSTRAINTS: Python 3.11, standard library only, no Node, no
npm, no classes unless needed.

INTERFACE (must match exactly):
  def find_doi(text: str) -> str | None
      # returns the first DOI found, lower-cased, or None
  def normalize_doi(raw: str) -> str
      # strips "https://doi.org/" or "doi:" prefixes and trailing
      # punctuation such as "." "," ")"

CONTEXT: A DOI looks like 10.1109/CVPR.2016.90 (prefix "10.",
4-9 digits, a slash, then a suffix). In PDFs it often appears
after the word "DOI" or inside a URL.

STEPS:
 1. Compile one regular expression for the DOI pattern.
 2. Implement find_doi using it.
 3. Implement normalize_doi.
 4. Add a small  if __name__ == "__main__":  block with 3 asserts.

OUTPUT FORMAT: Only the complete contents of doi_utils.py in a single
Python code block. No explanation before or after.
```

After Claude answers: *review it critically.* Does the file tree match the schema and API? Do all prompts respect the constraints? Does every prompt specify its interface? Does the plan accidentally contain Node, npm, React or a build step? Fix problems with a short follow-up, or edit the plan yourself. You are the engineer; Claude is your assistant.

6.3.4 Required content of `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`

```
# PLAN AND PROMPTS - Research Paper Library & Knowledge Graph
## A. Overview and key design decisions
## B. Technology stack (name | exact version | purpose | why | source URL)
## C. Architecture
   - File/folder tree with one-line purpose per file
   - Data flow: upload -> parse -> enrich -> tag -> store -> graph
## D. Database schema (CREATE TABLE statements)
## E. API contract (method | path | request | response | errors)
## F. Frontend specification (views, CDN tags with versions, graph JSON)
## G. Build order and dependencies
## H. Prompts for Qwen 2.5 Coder 7B   (Prompt 0 ... Prompt N)
   - each with: target file, role, constraints, interface, context,
     steps, output format, acceptance test
## I. Build log (filled in during Phase 4)
   | Prompt # | Attempts | Result | What I changed in the prompt |
## J. Manual edit log (filled in during Phase 4; may be empty)
   | File | What was changed | Why |
```

6.4 Phase 4 — Build the App with Qwen 2.5 Coder 7B

Goal: generate, run, test and debug the whole application using only Qwen, following your plan. **Tool:** Qwen 2.5 Coder 7B Instruct. **Output:** the working app (`Deliverable_4_APP_[HeUserID].zip`) plus the build and manual-edit logs in `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`.

6.4.1 The build loop

1. **Set up the skeleton yourself:** create the folder tree from your plan (empty files are fine), a virtual environment, and install the pinned libraries.
2. **Run Prompt *n* in a *fresh* chat.** Paste the prompt exactly as written in your plan. Fresh chats avoid context rot.
3. **Copy the code** into the right file.
4. **Run the module's acceptance test.** Do not move on until it passes.
5. **If it fails**, use the debugging protocol below. Record attempts in the build log.
6. **Commit or back up** the working state (a copy of the folder or a git commit) before starting the next module.
7. Repeat. Integrate the backend, then the frontend, then test the full flow with real PDFs.

6.4.2 Debugging protocol for a weak model

1. **Reproduce and isolate.** Find which file and which function fails. Read the traceback yourself first.
2. **Ask for a narrow fix.** Provide: the constraints (briefly), the *one* failing function or file, the *exact* error text, and what you expected. Ask for the corrected complete function/file only.
3. **Do not paste the whole project.** A 7B model gets worse with more text. Paste only what is needed.
4. **If it loops** (same wrong answer twice), start a fresh chat, restate the task, and add the missing detail that caused the confusion. Then *improve the original prompt in your plan* so it is right the first time. This feedback loop is the core skill of the assignment.
5. **Fix the cause, not the symptom.** If Qwen repeatedly gets something wrong, the usual reason is missing or ambiguous context; add a sample, a contract, or a rule.

Debugging prompt template

```
Constraints reminder: Python 3.11, SQLite, no Node/npm/frameworks.
Below is ONE function from backend/parser.py that fails.

[paste the function]

Error when I run it with a two-column PDF:
[paste exact traceback]

Expected behaviour: [one or two sentences]

Return ONLY the corrected complete function in one code block.
Do not change its name or signature.
```

6.4.3 Rules on manual edits

The purpose is to see what well-engineered prompts achieve with a weak model, so manual coding is not allowed to replace the model. You *may*: create folders and empty files, copy/paste generated code, rename files, install packages, change configuration values (ports, paths), and fix **trivial** issues such as an obvious typo or a wrong relative path (a few lines per file). **Every manual change must be listed** in the manual edit log (Section J of `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`). Large unlogged edits are treated as an integrity violation.

6.4.4 Recommended module order

A suggested (not mandatory) order that keeps each prompt small:

1. Database module: connection helper and table creation from your schema.
2. Data-access functions (insert/select for papers, authors, topics, references, activity log).
3. PDF text extraction module.
4. Metadata extraction: title/authors/abstract/keywords/DOI heuristics; references splitter.
5. Optional enrichment through a scholarly API, with a safe fallback if it fails.
6. Tagging module.
7. API routes (upload, list, detail, edit, delete, graph, activity).
8. Graph-building function that returns the JSON your frontend library expects.
9. Frontend: `index.html` skeleton and styling.
10. Frontend: library view, detail view, activity view.
11. Frontend: graph view.
12. Integration testing with 3–5 real PDFs; error-handling polish; README.

Context hygiene for small models

Keep a short running "project state" note (three or four lines: which files exist, their public function signatures, what works). Paste it at the top of a new Qwen chat when a module depends on earlier modules. It is your "memory bank" from the four pillars of context engineering.

6.5 Packaging the App

Your zip must run on someone else's machine by following your README. The grader will unzip it, create a fresh virtual environment, install your requirements, start the app with your documented command, and open it in a browser.

Required structure (folder names may differ, but the zip must contain a single project root):

```
Deliverable_4_APP_[HeUserID].zip
  project_root/
    README.md           <- how to install, run, and use (see below)
    requirements.txt    <- pinned Python dependencies
    backend/            <- Python code (entry point documented in README)
    frontend/           <- index.html, css/, js/  (no package.json!)
    data/               <- SQLite DB created on first run (may start empty)
    uploads/            <- uploaded PDFs (may start empty)
    sample_papers/      <- optional: 2-3 small open-access PDFs for testing
```

Do not include: virtual environments, `__pycache__`, `.git` folders, large files, or any Node

tooling.

Your `README.md` must contain: (1) a one-paragraph description; (2) Python version; (3) exact install commands; (4) the *single command* to start the app and the URL to open; (5) how to use each feature; (6) a short list of known limitations; (7) a statement of which AI model generated the code.

7 Deliverables and Submission

Submit exactly these four files, named with **strict** adherence to the following convention (replace [HeUserID] with your own user ID, keeping the brackets out only if the instructor says so; follow the course announcement):

#	File name	Contents
1	`Deliverable_1_CONTEXT_[HeUserID].md`	The sifted context pack from Phase 1 (Section 6.1).
2	`Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`	Architecture blueprint, stack table, schema, API contract, Qwen prompts, build log and manual-edit log (Phases 2–4).
3	`Deliverable_3_USER_PROMPTS_[HeUserID].md`	Verbatim log of every prompt you gave to AI tools in Phases 1–3, tagged with the tool; plus a short reflection.
4	`Deliverable_4_APP_[HeUserID].zip`	The runnable app (Phase 4), zipped.

Why are the plan's prompts and the user prompts separate files?

`Deliverable_3_USER_PROMPTS_[HeUserID].md` records the prompts *you gave to the research and planning AIs* (Gemini, DeepSeek, Perplexity, Claude): how you explored the problem. `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md` contains the prompts *for the final app* that you give to Qwen. The architecture itself lives inside `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md`, so there is no separate architecture file.

7.1 Required Format of `Deliverable_3_USER_PROMPTS_[HeUserID].md`

```
# USER PROMPTS LOG

## Entry 01
- Phase: 1 (Domain research)
- Tool / model / mode: Gemini - Deep Research   (or DeepSeek - DeepThink on)
- Date/time: 2025-..-.. 14:05
- Prompt (verbatim):
  > <paste the exact prompt you sent>
- Result (2-4 lines): what came back, how long, how useful
- What I kept / discarded and why: ...

## Entry 02
...

## Reflection (200-400 words)
1. Which prompt or piece of context made the biggest difference?
2. Where did Qwen fail, and what was missing from your context?
```

```
3. How would you structure your context differently next time?
4. One thing you learned about the strengths/limits of each tool.
```

Rules: prompts must be *verbatim* (copy-pasted, not summarized); include failed or abandoned prompts; keep entries in chronological order; if you uploaded files, name them.

7.2 Pre-Submission Checklist

- ☐ Four files, correct names, correct extensions.
- ☐ `Deliverable_1_CONTEXT_[HeUserID].md` has all 12 sections, real examples, sources, and a sifting log.
- ☐ `Deliverable_2_PLAN_AND_PROMPTS_[HeUserID].md` has every section A–J; every prompt has its acceptance test; the build log reflects what really happened.
- ☐ `Deliverable_3_USER_PROMPTS_[HeUserID].md` has verbatim prompts, tool names, and a reflection.
- ☐ The zip unpacks to a single project root; README works from a clean machine; no forbidden files (run the self-check from Section 5).
- ☐ The app starts with one command; upload → parse → library → graph works with at least three different PDFs.
- ☐ No API keys or passwords are hard-coded or included in the zip.

7.3 Submission Form

Kindly submit using your HEUSER Email on the following google form: https://forms.gle/hD744jWHbnNejqsA9

8 Grading

Component	Points	What earns the points
D1: Context pack	**25**	Coverage and *accuracy* of domain research (10); quality of sifting and the discard log (5); structure that an AI can use efficiently, real examples and sample JSON (5); sources cited (5).
D2: Plan & prompts	**25**	Complete architecture and file tree (6); correct, consistent schema and API contract (7); quality of the Qwen prompts against the checklist (8); acceptance tests, build log and manual-edit log (4).
D3: User prompts log	**15**	Completeness and verbatim honesty (6); right tool for each task (4); evidence of iterating and improving prompts (2); thoughtful reflection (3).
D4: Working app	**35**	Runs from the README on a clean machine (5); functionality F1–F12 (20: upload/parse 5, tags 3, DB & activity log 4, graph 5, library/detail UI 3); constraint compliance (5); code organization and README quality (5).
Total	**100**	
Bonus A	**+5**	Real sample papers given to the AI in Phase 1, with evidence and observed patterns used in the context pack.
Bonus B	**+3**	Extras such as duplicate detection by DOI, graceful offline fallback for metadata APIs, or simple automated tests.

Penalties

- **Forbidden technology in the app** (Node/Bun/Deno backend, npm, `package.json`, React/Vue/Angular, TypeScript, any build step): Deliverable 4 is reduced by 50%, and the constraint-compliance points are lost.
- **Wrong file names or format:** −5 points total.
- **App cannot be started** by following your README: the "functionality" score is capped at the part we can verify by reading the code.
- **Fabricated or edited-after-the-fact prompt logs, or undisclosed AI tools/manual coding:** academic-integrity process (see below).

9 AI-Use and Academic-Integrity Policy

This assignment *requires* AI tools, so the integrity rules are about **honesty and transparency**, not abstinence.

- You must use the assigned tools in the assigned roles and log them truthfully.
- Your logs must be real. We will compare your plan, your logs and your code for consistency; a prompt that could not have produced the code you submit is a red flag.
- Do not copy another student's context pack, plan or prompts. Discussing ideas is fine; submitting shared work is not.
- You are responsible for the facts in your context pack. AI tools make mistakes; verify claims against a primary source.
- Do not upload copyrighted material beyond what is necessary, and never submit secrets such as API keys.

10 Suggested Timeline

When	What	Hours
Days 1	Phase 1: research sessions, sifting, write `Deliverable_1_CONTEXT_[HeUserID].md`; start `Deliverable_3_USER_PROMPTS_[HeUserID].md`.	4–6
Day 2	Phase 2: Perplexity lookups, stack table.	1–2
Days 3	Phase 3: prepare inputs, planning prompt with Claude, review and fix the plan.	3–4
Days 4	Phase 4: build modules with Qwen, test, debug, log.	5–7
Days 5	Integration test with real PDFs, README, packaging, self-check, reflection.	2–3
Day 6	Final check against the checklist; submit.	1

11 Frequently Asked Questions and Common Pitfalls

Can I use Claude for everything? It is better than Qwen.
No. The pipeline and the weak-model constraint are the learning objective.

Qwen keeps ignoring my constraints.
Restate them at the top *and* the bottom of the prompt, make the prompt shorter, and ask for one file only. If it persists, split the task further.

Qwen's code works alone but modules do not fit together.
Your interface contracts were not explicit. Add exact function signatures and shared names to both prompts.

My PDF text comes out jumbled.
That is the PDF problem from Phase 1. Revisit how your chosen library orders text blocks, and see whether your sample papers use two columns.

A metadata API returned nothing or an error.
Real services fail. The app should fall back to the PDF-derived data and show a message, not crash.

I hit the free-tier limit.
Plan your usage, wait for the reset, or use the alternative tool in the same role. Document it in `Deliverable_3_USER_PROMPTS_[HeUserID].md`. Do not leave the work until the last day.

Can I use a database library or ORM?
Yes, as long as the database is SQLite and the backend is Python.

Can I use a CSS library?
Yes, from a CDN (for example a CSS-only framework). No build step or npm.

Is a login system required?
No. A simple user-name selector with server-side activity logging is enough for F6.

Can the app call an LLM API for extraction?
Not required, and keep in mind that free API tiers can change or disappear. If you use one, the app must still start without it, keys must be read from an environment variable (never committed), and your README must explain the setup. A fully local solution is perfectly acceptable.

12 Glossary

Agent	An LLM connected to tools and an execution environment that acts in a loop (read, edit, run, fix).
arXiv ID	Identifier of a preprint on arXiv, e.g. `2101.00001`.
CDN	Content Delivery Network; a public server from which the browser loads libraries via a URL.
Context window	The maximum number of tokens an LLM can consider at once.
Context engineering	Curating, structuring and managing the information given to an LLM.
CORS	Browser rule that blocks a web page from calling a different origin unless the server allows it.
DOI	Digital Object Identifier; a permanent ID for a paper, e.g. `10.1109/CVPR.2016.90`.
Hallucination	Confident but invented output (fake functions, facts or libraries).
LLM	Large Language Model.
Prompt engineering	Writing structured instructions to get predictable output from an LLM.
Reasoning model	An LLM that spends effort "thinking" step by step before answering.
Sifting	Removing noise from research so that only decision-relevant information remains.
Token	Basic unit of text for an LLM ($\approx$ 4 characters).
Vibe coding	Building software by describing intent to an AI and iterating on its output.

Final thought. The best engineers are not the ones who type the fastest; they are the ones who *understand the problem* and *communicate it precisely*. Whether the reader is a junior developer, a teammate or a 7B-parameter model, the skill is the same. Good luck!